Flags, consoles, pagination, master data

> *LORA's delivery.md asks four questions. Can you choose what deploys? Is a custom front end hard? Is pagination solved? Can master data fill dropdowns? We ask the same four of Bravo — and then look at what production says about its front ends.*

5	**15+**	**2.41M**	**1+1+0**
WAYS TO PICK WHAT RUNS. THERE IS NO REGISTER OF ANY OF THEM.	BROWSER APPLICATIONS AROUND BRAVO. LORA HAS TWO.	CONSOLE ERRORS IN 7 DAYS. THAT IS 5.8 TIMES LORA'S BACK OFFICE.	TO RENAME AN APPROVAL ROLE: ONE YAML LINE, ONE SQL UPDATE, NO JAVA AT ALL.

Audience Platform, front-end engineers, and product engineers shipping DF and NDF work **Sources** `bravo-bpm-service` v2.93.43 · Datadog us5 RUM and APM **Read** 2026-09-10

The four questions, plus the one nobody asked

Can you pick what deploys?	● YES — IN FIVE DIFFERENT WAYS	**And that is the problem.** Bravo has no LaunchDarkly either. What it has instead is five separate mechanisms: Spring property flags read *inside BPMN gateway conditions* (78 lookups), a six-column database selector, a per-application on/off matrix in jsonb, Camunda version pinning, and 31 Java factories. **There is no register, no owner, and no expiry date.** Turning a gateway flag on or off needs a configuration change *and a restart*, because it is an `environment.getProperty` call evaluated when the gateway runs.
Is a custom front end on a page hard?	● NO — AND THAT IS THE TRADE-OFF	Every Bravo console is an ordinary React app calling REST endpoints named after business actions. The interface never sees a Camunda task id. **One page is easy.** But there are **at least fifteen** separate browser applications, and a change to anything shared has to ship in every one of them that uses it. LORA has one back-office front end and a three-layer widget seam. **Bravo swapped a hard seam for a wide surface.**
Is pagination solved?	● YES — AND LORA'S ACCOUNT OF IT IS CONFIRMED	Confirmed from Bravo's own telemetry. Bravo built exactly the paging LORA describes: a controller literally named `SurveyorAssignmentFormTabV2Controller`, plus **nine** more endpoints for the parts of an assignment. Together they are the **busiest read path in the service.** LORA is right not to rebuild it, and Bravo was right that it needed it.
Can master data fill dropdowns?	● YES — THIS IS THE CAPABILITY LORA TURNED DOWN	LOV tables and three master-data services fill the dropdowns directly. That is why renaming an approval role (`BLCS-4683`, NMH to GMB) took **one YAML line, one SQL `UPDATE`, and no Java.** The cost is the one LORA designed around: the values are read *live*, so editing master data mid-flight can change what a form offers *after a decision has already been made from the old list.*
Are the front ends healthy?	● NO, AND NOBODY WAS LOOKING	Four monitored consoles produce **2,413,198 errors in 7 days** — 5.8 times LORA's back office. **748,686 of those are 404s** on the Surveyor Platform, and one endpoint reaches **69% of all sessions. No monitor can see any of it, because every health check only looks for status codes starting with 5.**

> **Every delivery mechanism Bravo needs exists, and most of them work. What is missing is a boundary.**
>
> No flag register, no list of consoles, no error budget for the front ends. So the delivery surface keeps growing, and nothing tells anyone that it has.

Feature flags: five mechanisms, no register, and a restart

LORA's complaint was "no feature flag; cannot pick what deploys", which it graded *partially true*. **Bravo fails in the opposite direction.** It has five ways to pick what runs, and nothing coordinates them.

MECHANISM	HOW FINE-GRAINED	TO CHANGE IT, YOU…	AFFECTS LOANS ALREADY RUNNING?
Spring property flags written inside BPMN gateway conditions — for example `environment.getProperty('setting.feature.config.featDF2W') == 'true'`**. There are 68 of these lookups in the older monoliths and 10 in the unified spine.**	The whole environment	change the configuration and **restart**, or refresh the config	Yes, the next time that gateway runs
Java service flags — `featRefactorWorkflow`, `featFinalApproverRoleOnReject`, `featSingleApproverRejectedRole`	The whole environment	the same	Yes
Six-column selector tables — `WorkflowProductConfig(productId, type, customerType, userType, businessType, riskType)`	Per combination of product, customer and risk	write a Flyway **data migration**	The next time the row is looked up
A per-application on/off matrix in jsonb — `{"CI": {"PilotBranchCheckActivity": [true,false]}}`**, read by** `BaseActivity.execute()` **before every unified activity**	**Per application, per activity**	it is worked out when the application starts, and worked out again mid-flight for the "SUO" phase	Fixed for each application when it starts
Camunda version pinning	Per process definition	deploy	The parent stays on its own version, but **each child picks the newest version when it is called**

WHAT IS GENUINELY GOOD HERE

The jsonb matrix is a **real per-application guard sitting on top of the flowchart**. It is the closest thing in either platform to LORA's `$.experiments.*`, and the team arrived at it independently. It has limits. It can only *skip* a step the diagram already contains, never add one or reorder them, and it is fixed per application rather than worked out from what data is ready. **But it is the right idea.**

A FLAG INSIDE A BPMN GATEWAY STRING IS NOT A RELEASE SWITCH

Flipping one is a **configuration deploy, not a switch**, and it applies to the whole environment at once. That is a *slower* release control than a code change behind a property, because the property now lives in XML as well. There is also **no register and no expiry date**: 78 gateway lookups, plus an open-ended set of Java flags, and no list of which ones are live. `featRefactorWorkflow` gates the whole unified-versus-legacy behaviour, and it has been in place since 2024.

THE REAL DEPLOY RISK IS NOT FLAGS. IT IS HOW CHILD PROCESSES ARE BOUND.

No `callActivity` sets `camunda:calledElementBinding`. So redeploying a sub-process changes the behaviour of loans that are *already running*, the next time they call it. That includes NDF2W loans entering unified underwriting. **That is 73% of production volume, with no migration plan.** It is the most important "what deploys to whom" question on this platform.

Custom front ends: easy on one page, hard across the system

LORA's complaint was that a custom widget takes **three layers** — templ emits a custom tag, SolidJS registers the `customElement`, HTMX carries the value back — and that templ cannot see neighbouring fields. Production backed that up: one script injected twice (8,067 times a week), and null errors from hydration running out of order about 34,900 times a week.

BRAVO HAS NONE OF THAT LAYERING

Every console is a plain React app calling REST endpoints named after the business action, such as `PATCH /v1/underwritings/{id}/approval/bm-decision`. The interface never sees a Camunda task id or a form definition. **Adding a field, a control, or a whole page is ordinary front-end work.**

THE COST TURNS UP SOMEWHERE ELSE: THE NUMBER OF FRONT ENDS

Anything shared — an approval banner, an asset-category field, a status label — has to be built and maintained **once in every console that shows it**. The `BLCS` tickets show it: `BLCS-4397` is *"Shared summary section config for both surfaces"*, `BLCS-4398` is *"Verify section matrix across 5 flows on both surfaces"*. **One story, spending all its effort on keeping two consoles in step.**

> **On a single page, Bravo is clearly easier. Across the whole system, it pays a coordination cost LORA does not —**
>
> exactly where its production errors are concentrated.

MEASURED 2026-09-10	SURVEYOR-CONSOLE	OPERATION-CONSOLE	UNDERWRITING-CONSOLE
`@bfi-finance/frontend-ui`	`3.0.155`	**`3.0.138`** — 17 versions behind	`^3.0.158`
React	`^17.0.2`	**`^18.2.0`**	`^17.0.2`
Vite	`7.3.6`	`7.3.6`	**`4.5.0`**
Linter · Node in CI	ESLint · 20	**Biome** · 20	ESLint · **18**
Lines of code · tests	306,670 · 286	207,825 · 102	249,366 · 441
Owning squad	`LN`	`LN`	`BLCS`

Three consoles, 763,861 lines of code, 829 tests — sharing a design system that has drifted into three versions, on two major versions of React. **This table is the bill for the coordination cost.**

CONSOLE IN THE BRAVO AND LOS ESTATE	BROWSER MONITORING
Surveyor Platform (Prod + Test)	**ALL**
Operation Platform (Prod + Test)	**ALL**
Underwriting Platform (Prod + Test)	**ALL**
Backoffice Web App	**ALL**
Supplier Web App	**ALL**
Customer Platform 4W · 2W · Sharia · DF · unified	**ALL**
Agency Cockpit	**NONE**
`prod-bravo-e-line-ui`, `prod-notary-ui`, `prod-bravo-lms-fe`, `prod-bravo-lms-fund-fe`, `Prod LMS Core FE`, `prod-bfi-dashboard-digital-fe`, `prod-bravo-edoc-download`, `PROD-DMS-FE`, `EGC Platform`, **agent and RO tooling**	mixed

At least **fifteen** browser applications. LORA has **two**: back office and customer.

Pagination: Bravo built it, called it `FormTab`, and it is now the busiest read path

CONTROLLER	CALLS / 2 DAYS	WHAT IT IS
PartnershipConfigurationController.getByApplicationId	**73,056**	Per-application configuration. Also the source of the 404 flood.
SurveyorAssignmentFormTabV2Controller.findByAssignmentId	**33,673**	**The paged form itself, version 2**
ApplicationDocumentTrackingController.get	32,820	Documents pane
SurveyorAssignmentManualKycController.scoring	30,591	KYC pane
SurveyorAssignmentController.exposeSurveyData	8,535	Survey payload
SurveyorAssignmentCompanyDataController.findByAssignmentId	7,526	Company pane
SurveyorAssignmentCompanyLegalityDocumentController	7,476	Legality pane
SurveyorAssignmentCapitalDetailController.findByAssignmentId	6,413	Capital pane
LongSurveyController.findAllByApplicationId / findByAssignmentId	5,077 / 4,903	Long-survey panes

Three more per-assignment endpoints appear in the 404 table as well: `surveyor-assignment-other-business`, `surveyor-assignment-asset-validation-two-wheeler` and `surveyor-assignment-request`.

LORA's delivery document grades this complaint **misstated**, and reasons as follows: *"In Bravo the surveyor saw all fields from all tasks on one giant form. They needed pagination by logical group. In LORA the surveyor opens one task and sees only that task's fields."* **That account is correct. And we can now check it against Bravo's own production data, instead of relying on memory.**

A CONTROLLER CALLED `FORMTABV2` IS PAGING THAT HAS ALREADY BEEN THROUGH ONE MAJOR REWRITE

Nine or more endpoints per assignment is exactly "pagination by logical group", built as **one endpoint per group**. It works, it is heavily used, and it is the main read path in `ms-bpm`.

BRAVO'S DESIGN IS NOT A WORKAROUND

It follows from what the screen shows. A Bravo surveyor opens an *assignment*, which covers the whole application. **Paging that is the only sensible thing to do, and calling it a defect misreads it.** LORA's recommendation also stands, and is now better evidenced: nobody should rebuild whole-application paging in LORA, because a LORA surveyor really does open one task at a time.

BUT IT HAS A REAL COST THAT LORA DOES NOT PAY

Opening one assignment sends **roughly ten HTTP requests**. Each is a controller, and each runs a set of JPA `repository.operation` queries against `prod-postgres-bpm-d2bpm` — which is 34.7% of Bravo's production Cloud SQL bill. And **four of those ten endpoints are the four biggest sources of the 404 flood**, so *the endpoints built for paging are also the endpoints that fail.*

Master data: Bravo has the feature LORA chose not to build

	BRAVO	LORA
Where dropdown options come from	LOV tables in the same database (`underwriting_job_level_lov`, `…_detail`, `surveyor_coverage`, `surveyor_level`), plus live calls to master-data services	A process-wide cache, loaded from BFI DMS when the worker starts; JSON Schema `enum` and `lov`; widget APIs
Master-data traffic, 7 days	`prod-ms-product` 31,463 · `prod-ms-master` 27,954 · `prod-ms-branch` 22,784 · `prod-ms-employee` 64,725	About 620,000 calls a week to `prod-ms-master` alone, through the gateway
Adding a new lookup	**Insert rows. The form reads them.**	An LGS inner proxy, a schema `lov`, and optionally a cache key — a change across several repositories
Renaming a role across the approval ladder	**One YAML line, one SQL `UPDATE`, and no Java** (`BLCS-4683`, NMH to GMB)	A schema change and a redeploy
Staying consistent for the life of the loan	**Not guaranteed.** Options are read live.	**Guaranteed.** Values are copied onto the document when it is written.

`BLCS-4683` IS THE STRONGEST SINGLE ARGUMENT FOR BRAVO'S APPROACH IN THIS PACK

Renaming an approval role across a *staged ladder* is the kind of change that normally ripples through code. Here it was a **configuration edit**. The ladder is only rows in `underwriting_job_level_lov_detail`, ordered by `global_level` — CAFH 200, then AMB 300, then CCU 400, then GMB 600 — and those values are copied onto approver rows. **That is data-driven delivery working exactly as intended.**

AND THE COST IS EXACTLY THE ONE LORA DESIGNED AROUND

Because options are read *live*, editing master data changes what a form offers straight away. That includes applications already in flight, and applications **already decided from the old list.** Bravo reduces the risk in one place: the approver ladder *is* copied onto approver rows when the approver is assigned. But that is **one path, done on purpose. It is not a property of the platform.**

THE RISK, PLAYING OUT IN THE TICKET STREAM RIGHT NOW

`BLCS-4683` renamed NMH to GMB in September 2026. `BLCS-4811`, raised **three days before this document**, is *"[BE] Prepare Query Change GMB to NMH"*. **The rename is being reversed.** A configuration change that is cheap to make is also cheap to make twice, and each edit lands on whatever applications happen to be in flight at that moment. **Nothing in Bravo records which applications were decided under which version of the ladder.** The master-data tables keep no history, and Camunda deletes its own history after 90 days.

Neither design is wrong. Bravo optimised for changing reference data quickly, and it got that. LORA optimised for keeping each loan's values fixed, and pays for it with a change across several repositories for every new lookup. The right answer differs from one lookup to the next — and neither platform lets you choose lookup by lookup.

2.4 million console errors a week, and nobody had looked

Sections 1 to 4 were written by reading code and configuration. **They did not have to be.** All four production Bravo LOS consoles already send browser monitoring data, at `rum_event_processing_state: ALL`. As with LORA's back office, nobody had opened the view.

CONSOLE	SESSIONS	VIEWS	ACTIONS	ERRORS	ERRORS PER VIEW
Surveyor Platform Prod	79,090	953,115	3,739,210	**1,962,909**	**2.06**
Operation Platform Prod	17,135	96,707	2,516,533	287,646	**2.97**
Underwriting Platform Prod	15,628	205,666	1,964,283	124,422	0.60
Customer Platform DF	2,594	31,412	87,473	38,221	1.22
Total, 7 days	**114,447**	**1,286,900**	**8,307,499**	**2,413,198**	**1.88**

For scale: LORA's back office produces 416,201 errors a week, about 2.8 per view, across 24,172 sessions. **Bravo's consoles produce 5.8 times as many errors, at a similar rate per view. Both teams had the monitoring, and neither was reading it.**

TOP ERRORS, SURVEYOR PLATFORM	PER 7 DAYS
`Request failed with status code 404`	**748,686**
`csp_violation` — `cdn-cgi/rum` blocked by `connect-src`	35,415
`intervention: Ignored attempt to cancel a touchmove event` — tablet devices	20,928
`Request failed with status code 400`	12,796
The same beacon, from an origin with an explicit port	10,573
`Error getting location: "[GeolocationPositionError]"`	**10,365**
`Unable to get current position` — the same defect under a second message	**10,325**
`Request failed with status code 401` — the browser side of 8,533 IAM 401s	7,402
`Network Error`	6,828
`Cannot read properties of null (reading 'filter')` — a genuine null error	6,445

This is the console behind `Surveyor Platform - Release reject`, which is **28.4% of Bravo's entire support-ticket load.**

Caveats. Browser sessions are sampled, and the counts include harmless browser noise. The two CSP rows, 45,988 between them, come from a third-party beacon and are cosmetic, and `utc_offset is deprecated` (5,950) is a library warning. The rows worth acting on are the 404s, the location errors and the null error. The window is the 7 days to 2026-09-10.

§5 / THREE FINDINGS THAT CHANGE THIS DOCUMENT

The endpoints built for paging are the endpoints that fail

A · THE 404S ARE LANDING ON THE SAME PAGING ENDPOINTS AS §3

ENDPOINT	404S / 7 DAYS	SESSIONS
/bpm/v1/partnership-configuration/feature-configuration	266,767	54,350 of 79,090 — 69%
/bpm/v2/surveyor-assignment-form-tab/assignment/{id}	157,623	37,359
/bpm/v1/application-document-tracking	154,097	45,916
/bpm/v1/surveyor-assignment-manual-kyc/scoring/assignment/{id}	143,180	31,565
/surveyor/assignment-detail/{id}	21,125	14,754
/surveyor/assignment	14,565	11,287

Three of the top four are the assignment endpoints — the `FormTab` paging model. The first is the busiest handler in the entire service, and **two-thirds of all surveyor sessions get a 404 from it.** It may be harmless: it could be an optional configuration probe, where 404 simply means "no override". Or it could be a defect. Only the controller can settle that; telemetry cannot. But at this volume, on this console, somebody has to ask. *It is section 3's mechanism and section 5's largest error, and nobody had connected them, because nobody had opened either view.*

B · LOCATION CAPTURE FAILS ABOUT 20,700 TIMES A WEEK, ON A PLATFORM SPENDING RP64.0M A MONTH ON MAPS

> `GeolocationPositionError` and `Unable to get current position` are **the same defect, reported twice**. Surveyors work in the field, and their submissions depend on capturing a position. Separately, bravo-cost.md §5 finds that Geocoding, Places and Maps billed **Rp63,979,414 in August**. We are not claiming one causes the other — a browser failure and a server-side API call are different layers. **But they are the same feature, in the same console, and nobody has examined either.**

C · NO ALERT CAN SEE ANY OF THIS

```
(hits − hits{http.status_code:5*}) / hits < 99
```

> By that definition a 404 counts as a *success*, and **not one of the 38 monitors looks at browser data at all.** This is the delivery side of the same finding: **Bravo can ship a console change that returns 404 to two-thirds of sessions, and every signal it owns stays green.**

HOW TO REPRODUCE

```
# Datadog RUM, Surveyor Platform Prod
@application.id:2f3ca103-2053-4601-8e2a-5de3aee713bb
  @type:error
  → group by @error.message

# the 404 endpoints
@application.id:2f3ca103-… @type:resource
  @resource.status_code:404
  → group by @resource.url_path_group,
    count(*) and cardinality(@session.id)
```

Nine actions

1 **Work out what the `feature-configuration` 404 means** · S, DO THIS FIRST

266,767 a week, reaching 69% of surveyor sessions, on the busiest handler in the service. Decide whether 404 is the response we intend for "no override". If it is,
stop the client treating it as an error
, because right now it is drowning that console's error stream. If it is not, this is a live production defect, and it has been invisible for as long as we have data.

2 **Fix the location failures, and check them against the Maps bill** · S–M

About 20,700 failures a week, on a console used by field staff. Find the cause — permissions, HTTPS context, device, or timeout — and find out whether the retries are billed against the Rp64.0M a month Maps line.

3 **Set an error budget for each console** · S

The Surveyor Platform runs at 2.06 errors per view today, and Operation at 2.97.
If either doubled, no number anywhere would move.
One SLO per console, reviewed weekly, is the cheapest thing on this list.

4 **Filter out the CSP beacon rows** · S

45,988 `cdn-cgi/rum` violations a week are cosmetic, and they are the second- and fifth-largest rows on the Surveyor Platform. Either allow the origin in `connect-src`, or stop loading the beacon.
As things stand, they hide real errors.

5 **Build a feature-flag register** · S–M

There are 78 gateway lookups, plus the Java flags, plus the selector tables. There is no list of which are live, what each one gates, who owns it, or when it can be removed.
Start by listing them. The ones to delete will be obvious.

6 **Pin `calledElementBinding`, or decide deliberately to move running loans** · S–M

This is the real "choose what deploys" question on Bravo, and
right now it is answered by default.
Also listed as recommendation 1 in bravo-testing.md.

7 **List the consoles, and name an owner for shared components** · M

There are at least fifteen browser applications, and stories are already spending their effort on "both surfaces". Either build a shared component library and give it an owner, or decide explicitly that each console goes its own way.

8 **Give reference data a version, or copy it onto the application** · M

The NMH-to-GMB rename, and the reversal now in progress, show a configuration change cheap enough to make twice. After 90 days you cannot tell which applications were decided under the old ladder and which under the new one. Either keep versions of the LOV tables, or copy the values onto the application, the way approver rows already do.

9 **Add monitoring to Agency Cockpit, or retire it** · S

It is the only console near the LOS with `rum_event_processing_state: NONE`, so nothing is watching it.

30, 60, 90 days

Nine recommendations, phased against **about 25 person-days a month** of Squad S&U time, plus a slice of Platform and SRE.

THE RULE FOR ORDERING THIS WORK

Fix what users hit before rebuilding what engineers dislike. The 404 flood reaches 69% of surveyor sessions, on the console that generates 28.4% of Bravo's support tickets. The console inventory and the shared-component question matter structurally, but they *hurt nobody today*. So the live defects go first, and the shape of the delivery surface goes last.

SHARED WITH OTHER DOCUMENTS

Recommendation 1 is also rec 1 in **bravo-observability.md** and rec 5 in **bravo-cost.md**. Recommendation 2 goes with rec 3 in **bravo-cost.md**. Recommendation 3 goes with rec 10 in **bravo-observability.md**. Recommendation 6 *is* rec 1 in **bravo-testing.md**. Phased the same way everywhere, so do them once.

Days 0–30

THE LIVE DEFECTS AND THE CHEAP NOISE

1 **Work out what the `feature-configuration` 404 means** — is 404 the intended "no override" response, or a defect? *S&U · 2 days.* A written answer, for the endpoint that hits 54,350 of 79,090 sessions.

1 **Act on the answer** — fix the lookup, or stop the client treating it as an error. *S&U · 3 days.* The Surveyor Platform's error stream drops by roughly a third, and whatever the 404 was hiding becomes visible.

4 **Filter or allow the CSP beacon** — 45,988 violations a week. *Platform · 1 day.* Real errors stop being buried.

9 **Add monitoring to Agency Cockpit, or retire it.** *Platform · 1 day.*

6 **Investigate `calledElementBinding` — read-only** *(shared). S&U · 5 days.* A written note on the exposure. **No code change.**

Days 31–60

BUDGETS, THE FIELD DEFECT, THE FLAG LIST

2 **Fix the location failures** — about 20,700 a week on a console used by field staff. Find the cause: permissions, HTTPS context, device, or timeout. *S&U · 5 days.* The cause is documented and checked against the Rp64.0M a month Maps line.

3 **Set an error budget for each console** — one SLO each, reviewed weekly. *Platform, SRE and EM · 2 days.* A console regression gets caught by a number, instead of by 315 `Release reject` tickets a month.

5 **Build the feature-flag register** — what is live, what it gates, who owns it, and when it expires. *S&U · 4 days.* Somebody can answer "what does `featRefactorWorkflow` gate, and can we delete it?" without reading 53 BPMN files.

6 **Decide `calledElementBinding`** *(shared). S&U and EM · 2 days.* A decision on the record, with a migration plan.

Days 61–90

THE DELIVERY SURFACE ITSELF

6 **Carry out the binding decision** *(shared; the effort is carried in bravo-testing.md).* Redeploying a unified sub-process stops silently changing the path of NDF2W loans that are already running.

7 **List the consoles and name an owner for shared components** — at least fifteen browser applications, with stories already spending their effort on "both surfaces". *EM and the front-end leads · 3 days.* Either a shared component library with an owner, or an explicit decision, on the record, that the consoles go their own way.

Deferred

WITH TRIGGERS

7 **The shared component library itself**, beyond the list. An organisational decision with a budget attached, not an engineering task. *Trigger: the phase-3 inventory.*

8 **Version the reference data, or copy it onto the application.** 10 days. The design depends on the Camunda history decision in **bravo-cost.md rec 4**, because both are about what we can still know after day 91. *Trigger: that decision landing.*

What changes when this is done

RECOMMENDATION	EXAMPLE WHEN DONE
`feature-configuration` 404 resolved	The Surveyor Platform's error stream drops by roughly a third, and whatever the 404 was hiding becomes visible.
Location capture fixed	Field surveyors stop failing to capture a position about 20,700 times a week, and the Maps bill gets an explanation.
Error budgets for the front ends	A console regression gets caught by a number, instead of by 315 `Release reject` tickets a month.
Flag register	Somebody can answer "what does `featRefactorWorkflow` gate, and can we delete it?" without reading 53 BPMN files.
`calledElementBinding` decided	Redeploying a unified sub-process stops silently changing the path of NDF2W loans that are already running.
Console ownership	"Shared summary section config for both surfaces" becomes a change to one component, not a story.
Reference data versioned	The question "which ladder was this application approved under?" has an answer at day 120.

> **Bravo can ship a console change that returns 404 to two-thirds of sessions, and every signal it owns stays green. One SLO per console fixes that.**